

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

А. В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 4

ЗАПРОСЫ НА ЯЗЫКЕ SQL: АГРЕГАТНЫЕ ФУНКЦИИ

по курсу:

МЕТОДЫ И СРЕДСТВА ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННЫХ
СИСТЕМ И ТЕХНОЛОГИЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4326

подпись, дата

Г. С. Томчук
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: изучение применения агрегатных функций и теоретико-множественных операций в MySQL для анализа данных.

2 Постановка задачи

Требуется реализовать запросы с агрегатными функциями (пункты г–е варианта), дополнительно разработать собственные запросы с агрегатами, а также продемонстрировать использование операций UNION, INTERSECT, EXCEPT и сравнить их поведение с операциями над мультимножествами.

Вариант: 4.

Создайте базу данных для хранения следующих сведений: студент, группа, дисциплина, преподаватель, лабораторная работа, рейтинг за сданную лабораторную работу.

Составить запросы на:

1. количество работ, сданных каждым студентом по БД;
2. студентов, у которых средний рейтинг за сданные лабораторные работы по БД превышает 4;
3. студентов, не сдавших ни одной работы по БД.

3 Ход выполнения

В ходе выполнения работы была использована ранее созданная база данных, содержащая сведения о студентах, дисциплинах, лабораторных работах и их результатах.

На первом этапе были проверены данные, при необходимости добавлены недостающие записи для корректной работы агрегатных запросов. Созданный SQL-скрипт приведен в Приложении.

Далее были реализованы запросы по варианту задания:

1. На рисунке 1 приведен код запроса (г) и результат его выполнения (количество работ, сданных каждым студентом по БД).

```

25 -- г) количество работ по Базам данных для каждого студента
26 SELECT
27     s.student_id,
28     COUNT(*) AS work_count
29 FROM
30     student s
31     JOIN lab_rating lr ON s.student_id = lr.student_id
32     JOIN lab_work lw ON lr.lab_id = lw.lab_id
33     JOIN discipline d ON lw.discipline_id = d.discipline_id
34 WHERE
35     d.discipline_name = 'Базы данных'
36 GROUP BY
37     s.student_id;
38
39 -- д) студенты со средним рейтингом > 4

```

student_id	work_count
1	2
3	1

Рисунок 1 — Вычисление кол-ва сданных работ по БД для каждого студента

2. На рисунке 2 приведен код запроса (д) и результат его выполнения (студенты со средним рейтингом выше 4).

```

39 -- д) студенты со средним рейтингом > 4
40 SELECT
41     s.student_id,
42     AVG(lr.score) AS avg_score
43 FROM
44     student s
45     JOIN lab_rating lr ON s.student_id = lr.student_id
46     JOIN lab_work lw ON lr.lab_id = lw.lab_id
47     JOIN discipline d ON lw.discipline_id = d.discipline_id
48 WHERE
49     d.discipline_name = 'Базы данных'
50 GROUP BY
51     s.student_id
52 HAVING
53     AVG(lr.score) > 4;
54
55 -- е) студенты без работ по Базам данных

```

student_id	avg_score
1	4.5

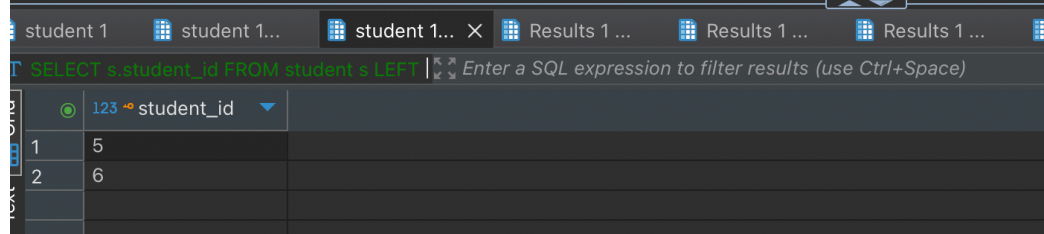
Рисунок 2 — Выборка по среднему рейтингу студентов

3. На рисунке 3 приведен код запроса (е) и результат его выполнения (студенты, не сдавшие ни одной работы по БД).

```

55 -- е) студенты без работ по Бадам данных
56 SELECT
57     s.student_id
58 FROM
59     student s
60     LEFT JOIN lab_rating lr ON s.student_id = lr.student_id
61     LEFT JOIN lab_work lw ON lr.lab_id = lw.lab_id
62     LEFT JOIN discipline d ON lw.discipline_id = d.discipline_id
63     AND d.discipline_name = 'Базы данных'
64 WHERE
65     d.discipline_id IS NULL;
66
67 -- дополнительные агрегаты

```



	student_id
1	5
2	6

Рисунок 3 — Выборка студентов, не сдавших ни одной работы по БД

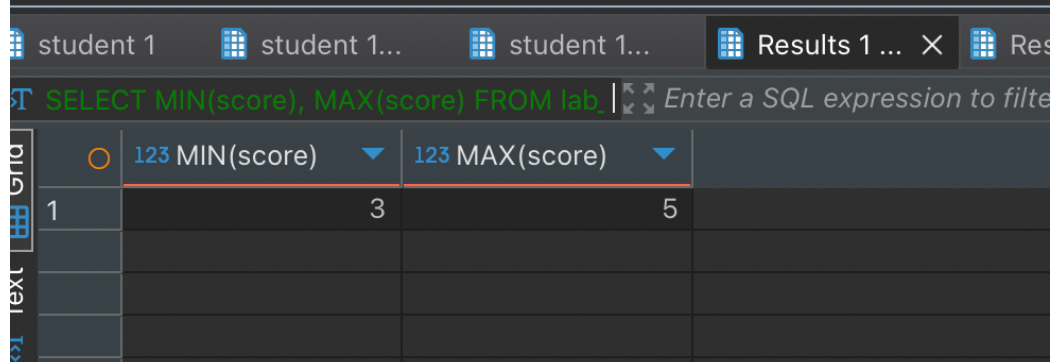
Затем были разработаны дополнительные запросы с агрегатными функциями MIN, MAX, и AVG:

1. На рисунке 4 приведен пример использования MIN и MAX.

```

67 -- дополнительные агрегаты
68 SELECT
69     MIN(score),
70     MAX(score)
71 FROM
72     lab_rating;
73

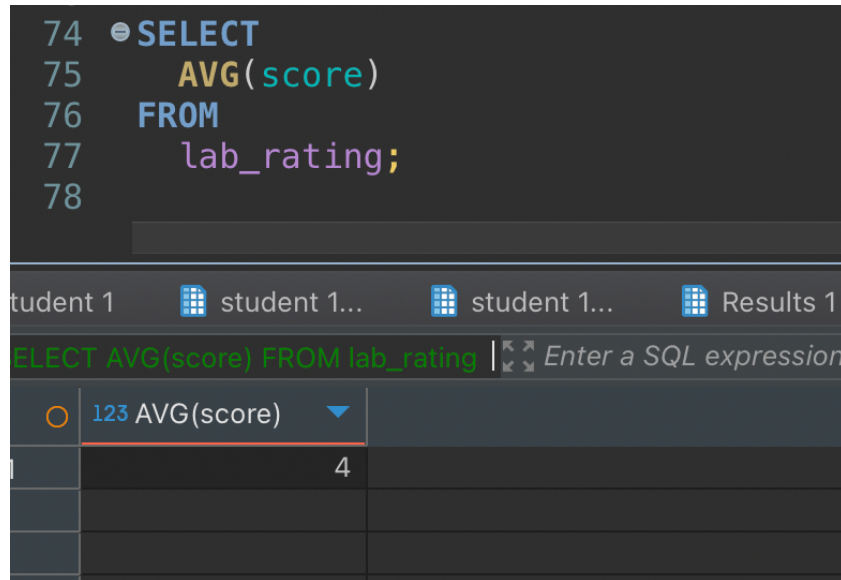
```



	123 MIN(score)	123 MAX(score)
1	3	5

Рисунок 4 — Минимальная и максимальная выставленная оценка

2. На рисунке 5 приведен пример использования AVG.



```
74 SELECT
75     AVG(score)
76 FROM
77     lab_rating;
78
```

student 1 student 1... student 1... Results 1

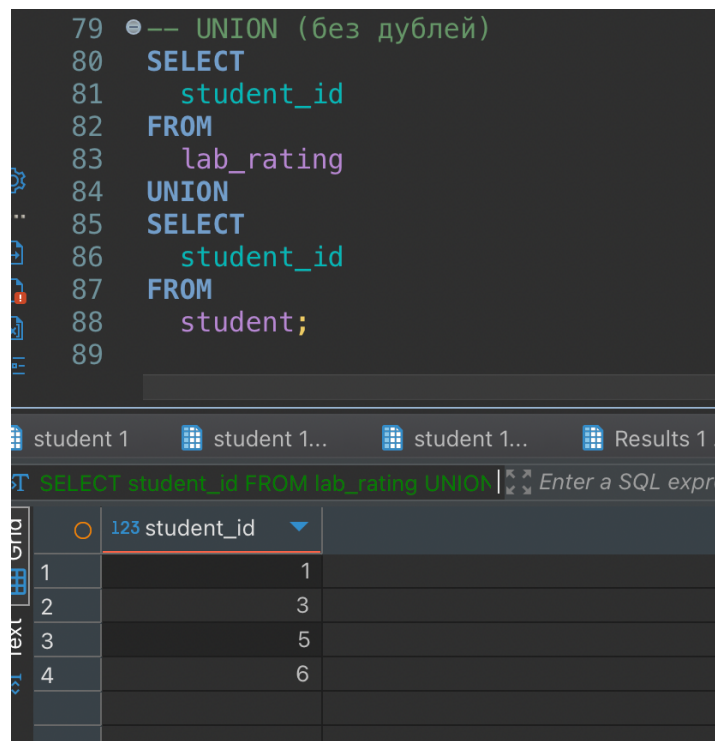
SELECT AVG(score) FROM lab_rating | Enter a SQL expression

	123 AVG(score)
	4

Рисунок 5 — Средняя оценка по всем работам всех студентов

После этого были реализованы теоретико-множественные операции:

1. На рисунке 6 приведен пример использования UNION.



```
79 -- UNION (без дублей)
80 SELECT
81     student_id
82 FROM
83     lab_rating
84 UNION
85 SELECT
86     student_id
87 FROM
88     student;
89
```

student 1 student 1... student 1... Results 1

SELECT student_id FROM lab_rating UNION | Enter a SQL expression

	123 student_id
1	1
2	3
3	5
4	6

Рисунок 6 — Объединение ID студентов из таблиц студентов и оценок
теоретико-множественным способом UNION

2. На рисунке 7 приведен пример использования UNION ALL
(мультимножество).

```

90 -- UNION ALL (с дублями)
91 SELECT
92     student_id
93 FROM
94     lab_rating
95 UNION ALL
96 SELECT
97     student_id
98 FROM
99     student;
100

```

student_id
1
1
3
1
5
6
3

Рисунок 7 — Объединение ID студентов из таблиц студентов и оценок мультимножественным способом UNION ALL

3. На рисунке 8 приведен пример использования INTERSECT.

```

101 -- INTERSECT
102 SELECT
103     student_id
104 FROM
105     student INTERSECT
106 SELECT
107     student_id
108 FROM
109     lab_rating;
110

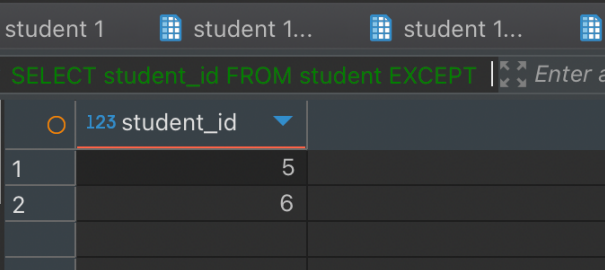
```

student_id
1
3

Рисунок 8 — Пересечение множеств ID студентов из таблиц студентов и оценок (студенты, которые сдали хотя бы одну работу)

4. На рисунке 9 приведен пример использования EXCEPT.

```
111 -- EXCEPT
112 SELECT
113     student_id
114 FROM
115     student EXCEPT
116 SELECT
117     student_id
118 FROM
119     lab_rating;
120
```



	123 student_id	
1	5	
2	6	

Рисунок 9 — Исключение ID студентов lab_rating из student (студенты, которые не сдали ни одной работы)

В ходе анализа было установлено различие между операциями множеств и мультимножеств: при использовании операций над множествами (UNION) производится устранение дубликатов (дедупликация), тогда как операции над мультимножествами (UNION ALL) сохраняют все записи, включая повторяющиеся. Удаление дубликатов требует дополнительных вычислительных затрат (сортировки или хеширования), поэтому операции над множествами, как правило, менее производительны по сравнению с операциями над мультимножествами.

4 Выводы

В ходе выполнения работы были освоены агрегатные функции SQL (COUNT, AVG, MIN, MAX) и их применение для анализа данных. Были реализованы запросы с группировкой и условиями отбора.

Дополнительно изучены теоретико-множественные операции (UNION, INTERSECT, EXCEPT) и установлены различия между операциями над множествами и мультимножествами.

Получен практический опыт обработки данных и формирования аналитических запросов в реляционной базе данных.

ПРИЛОЖЕНИЕ

```
USE university_db;

-- добавление данных для корректных выборок
/*INSERT INTO
student (
last_name,
first_name,
birth_date,
group_id,
email
)
VALUES
(
'Тестов',
'Иван',
'2000-01-01',
1,
'test@mail.com'
);

INSERT INTO
lab_rating (student_id, lab_id, score, submission_date)
VALUES
(1, 1, 5, '2024-01-01');*/
-- г) количество работ по Базам данных для каждого студента
SELECT
s.student_id,
COUNT(*) AS work_count
FROM
student s
JOIN lab_rating lr ON s.student_id = lr.student_id
JOIN lab_work lw ON lr.lab_id = lw.lab_id
JOIN discipline d ON lw.discipline_id = d.discipline_id
WHERE
d.discipline_name = 'Базы данных'
GROUP BY
s.student_id;

-- д) студенты со средним рейтингом > 4
SELECT
s.student_id,
AVG(lr.score) AS avg_score
FROM
student s
JOIN lab_rating lr ON s.student_id = lr.student_id
JOIN lab_work lw ON lr.lab_id = lw.lab_id
JOIN discipline d ON lw.discipline_id = d.discipline_id
WHERE
d.discipline_name = 'Базы данных'
GROUP BY
s.student_id
HAVING
AVG(lr.score) > 4;

-- е) студенты без работ по Базам данных
SELECT
s.student_id
FROM
student s
LEFT JOIN lab_rating lr ON s.student_id = lr.student_id
```



```

LEFT JOIN lab_work lw ON lr.lab_id = lw.lab_id
LEFT JOIN discipline d ON lw.discipline_id = d.discipline_id
AND d.discipline_name = 'Базы данных'
WHERE
    d.discipline_id IS NULL;

-- дополнительные агрегаты
SELECT
    MIN(score),
    MAX(score)
FROM
    lab_rating;

SELECT
    AVG(score)
FROM
    lab_rating;

-- UNION (без дублей)
SELECT
    student_id
FROM
    lab_rating
UNION
SELECT
    student_id
FROM
    student;

-- UNION ALL (с дублями)
SELECT
    student_id
FROM
    lab_rating
UNION ALL
SELECT
    student_id
FROM
    student;

-- INTERSECT
SELECT
    student_id
FROM
    student INTERSECT
SELECT
    student_id
FROM
    lab_rating;

-- EXCEPT
SELECT
    student_id
FROM
    student EXCEPT
SELECT
    student_id
FROM
    lab_rating;

```